

AGUACATIA: Sistema de Clasificación Visual del Estado de Maduración del Aguacate Hass mediante Visión por Computador

Documentación Técnica de Proyecto de Maestría

María Alejandra Gómez Piedrahita

Juan Manuel Castillo Pinto

Bogotá D.C., Colombia — Junio 2026

Universidad de La Salle

Facultad de Ingeniería

Maestría en Ciencias de la Computación

Proyecto: AGUACATIA — Sistema de Clasificación Visual del Estado de Maduración del Aguacate Hass mediante Visión por Computador

Autores:

- María Alejandra Gómez Piedrahita
- Juan Manuel Castillo Pinto

Asignatura / Proyecto de grado: Maestría en Ciencias de la Computación

Fecha de entrega: Junio 2026

Repositorio: <https://github.com/jmmana/aguacatia>

Aplicación en producción: <https://aguacatia.warlockcode.com>

Resumen

El presente documento describe el diseño, implementación y despliegue de Aguacatia, un sistema de inteligencia artificial para la clasificación visual automatizada del estado de maduración del aguacate Hass (*Persea americana* cv. Hass). El sistema utiliza YOLOv8n-cls, una red neuronal convolucional de última generación entrenada sobre el *Hass Avocado Ripening Photographic Dataset* (14.710 imágenes, 5 clases de maduración), para identificar en tiempo real el estado del fruto a partir de fotografías tomadas con dispositivos móviles convencionales.

La arquitectura del sistema comprende: un pipeline de entrenamiento en Google Colab con GPU NVIDIA, una API REST desarrollada en FastAPI (Python 3.11) para servir las predicciones del modelo, una interfaz web accesible desde cualquier navegador, y un volumen de almacenamiento persistente sobre SQLite. El despliegue en producción se realizó sobre infraestructura Coolify con Traefik como proxy inverso y Cloudflare como CDN/DNS.

Los resultados muestran que el modelo entrenado con GPU T4 alcanzó una precisión de validación del 82,7% en las 5 etapas de maduración. El sistema está completamente operativo y accesible en <https://aguacatia.warlockcode.com>.

Palabras clave: visión por computador, clasificación de imágenes, aguacate Hass, YOLOv8, FastAPI, maduración de frutas, inteligencia artificial aplicada.

Abstract

This document describes the design, implementation and deployment of Aguacatia, an artificial intelligence system for the automated visual classification of Hass avocado (*Persea americana* cv. Hass) ripening stages. The system employs YOLOv8n-cls, a state-of-the-art convolutional neural network trained on the *Hass Avocado Ripening Photographic Dataset* (14,710 images, 5 ripeness classes), to identify the fruit's maturity stage in real time from photographs taken with standard mobile devices.

The system architecture includes: a GPU-accelerated training pipeline on Google Colab, a FastAPI REST API (Python 3.11) to serve model predictions, a browser-accessible web interface, and a SQLite-backed persistent storage volume. Production deployment was carried out on Coolify infrastructure with Traefik as a reverse proxy and Cloudflare as CDN/DNS provider.

Results show that the model trained on a T4 GPU achieved 82.7% validation accuracy across 5 ripeness stages. The system is fully operational at <https://aguacatia.warlockcode.com>.

Keywords: computer vision, image classification, Hass avocado, YOLOv8, FastAPI, fruit ripeness, applied artificial intelligence.

Tabla de contenido

1. Introducción
 2. Marco Teórico
 3. Dataset
 4. Arquitectura del Sistema
 5. Metodología y Entrenamiento del Modelo
 6. API REST
 7. Interfaz Web
 8. Métricas y Evaluación
 9. Despliegue en Producción
 10. Conclusiones y Trabajo Futuro
 11. Referencias Bibliográficas
-

1. Introducción

1.1 Contexto del problema

La evaluación del estado de maduración del aguacate Hass (*Persea americana* cv. Hass) se realiza actualmente de manera manual, basándose en la experiencia visual de agricultores, comercializadores y consumidores. Este proceso es subjetivo, inconsistente entre evaluadores y poco escalable para operaciones de alto volumen como las de fincas productoras o supermercados.

Colombia es uno de los principales productores mundiales de aguacate Hass, con creciente demanda tanto en el mercado nacional como en exportaciones hacia Europa, Estados Unidos y Asia. En 2023, Colombia exportó más de 140.000 toneladas de aguacate Hass, consolidándose como el tercer exportador mundial después de México y Perú. Una clasificación visual precisa y automática representa un valor directo para toda la cadena productiva: productores, exportadores, distribuidores y consumidores finales.

El problema central es que dos aguacates del mismo peso y tamaño pueden estar en estados de maduración completamente distintos, y determinar esta diferencia requiere experiencia visual que solo ciertos expertos poseen. En cadenas de distribución donde el tiempo es crítico, un sistema automatizado que clasifique el estado del fruto desde una fotografía puede reducir pérdidas, optimizar inventarios y mejorar la experiencia del consumidor final.

1.2 Motivación

El proyecto surge de una necesidad real y directa: dos integrantes del equipo son propietarios de fincas productoras de aguacate Hass en Colombia. Esta condición garantiza:

- Acceso continuo a material visual auténtico y variado (distintas condiciones de luz, fondos, ángulos)

- Comprensión profunda del problema desde la perspectiva del productor
- Posibilidad de validación del sistema en condiciones reales de campo
- Retroalimentación directa sobre la utilidad práctica del sistema

La pregunta que motivó el proyecto es simple: *¿puede un productor o distribuidor fotografiar un aguacate con su celular y obtener en segundos una clasificación objetiva de su estado de maduración?*

1.3 Pregunta de investigación

¿Es posible construir un sistema de visión por computador basado en redes neuronales convolucionales que clasifique con alta precisión el estado de maduración del aguacate Hass a partir de fotografías tomadas con dispositivos móviles convencionales?

1.4 Objetivo general

Desarrollar e implementar un sistema de clasificación de imágenes capaz de identificar visualmente el estado de maduración de un aguacate Hass a partir de fotografías tomadas desde un dispositivo móvil, con retroalimentación en tiempo real al usuario, y desplegarlo como servicio web accesible en producción.

1.5 Objetivos específicos

1. Preparar un dataset de imágenes de aguacates Hass etiquetado con las 5 etapas de maduración para entrenamiento supervisado.
2. Entrenar un modelo de clasificación YOLOv8n-cls sobre las 5 clases de maduración y optimizar sus hiperparámetros.
3. Evaluar el modelo entrenado con métricas estándar de clasificación multiclase: accuracy, precision, recall, F1-score, matriz de confusión, curvas ROC y AUC.
4. Implementar una API REST en FastAPI que sirva las predicciones del modelo con baja latencia y persistencia del historial.
5. Desarrollar una interfaz web accesible desde cualquier navegador, sin requerir instalación de software adicional.
6. Desplegar el sistema completo en infraestructura en la nube con CI/CD automatizado.

1.6 Alcance y limitaciones

Dentro del alcance: El sistema realiza una *estimación visual* del estado de maduración basada exclusivamente en características externas observables:

- Color de la piel (verde amarillento → verde oliva → manchas púrpuras → púrpura uniforme → deterioro)
- Textura superficial
- Presencia de manchas o signos de deterioro
- Brillo y uniformidad de la superficie

Fuera del alcance:

- Determinación de maduración interna (contenido de grasa, firmeza, etileno)
- Clasificación de variedades distintas al aguacate Hass
- Reconocimiento de enfermedades o plagas específicas
- Procesamiento de video en tiempo real

Limitaciones conocidas:

- El dataset de entrenamiento fue recolectado en condiciones controladas de laboratorio (iluminación LED, fondo blanco uniforme), lo que genera una diferencia de dominio con fotografías tomadas en campo real.
- Las etapas adyacentes de maduración (particularmente *Breaking* y *Ripe First Stage*) tienen características visuales similares y presentan mayor tasa de error esperada.
- El modelo no distingue el estado de maduración interno del fruto; un aguacate puede presentar aspecto externo de una etapa y estado interno diferente.

1.7 Organización del documento

Este documento sigue el ciclo completo del proyecto, desde los fundamentos teóricos hasta el despliegue en producción:

Capítulo	Contenido
2	Marco teórico: visión por computador, CNNs, YOLO, transfer learning
3	Dataset: fuente, composición, preparación, split
4	Arquitectura del sistema: componentes, flujo de datos, decisiones de diseño
5	Metodología y entrenamiento: configuración, aumentos, resultados experimentales
6	API REST: endpoints, flujo interno, estructura de archivos
7	Interfaz web: páginas, flujo del usuario, tecnologías
8	Métricas y evaluación: metodología, resultados, análisis de errores
9	Despliegue: infraestructura, CI/CD, URLs de producción
10	Conclusiones y trabajo futuro

2. Marco Teórico

2.1 La industria del aguacate Hass en Colombia

El aguacate Hass (*Persea americana* var. Hass) es la variedad comercial dominante a nivel mundial, representando más del 80% de la producción global de aguacate destinada a exportación. Sus características distintivas — piel rugosa que cambia de color durante la maduración, alto contenido de grasa monoinsaturada y vida útil prolongada — lo hacen ideal para la cadena de distribución internacional.

Colombia se posiciona estratégicamente como productor de aguacate Hass gracias a sus condiciones agroecológicas excepcionales: altitudes entre 1.800 y 2.500 m.s.n.m. en los departamentos de Antioquia, Caldas, Risaralda y Valle del Cauca permiten producción escalonada durante todo el año, sin la estacionalidad característica de los principales competidores (México y Perú).

La identificación precisa del estado de maduración es crítica en tres puntos de la cadena:

1. **En finca:** determinar el momento óptimo de cosecha para cada lote
2. **En centro de acopio:** clasificar los frutos para distribución diferenciada según destino (consumo inmediato vs. almacenamiento vs. exportación)
3. **En punto de venta:** informar al consumidor final sobre el tiempo esperado de maduración

La clasificación manual actual depende de la experiencia subjetiva del evaluador y varía entre regiones, empresas y personas, generando inconsistencias que se traducen en pérdidas estimadas del 10-15% por frutos cosechados fuera de la ventana óptima.

2.2 Visión por computador en agricultura

La visión por computador aplicada al sector agrícola (*precision agriculture* o *agriculture 4.0*) es un área de investigación activa desde la década de 2010. Las aplicaciones incluyen:

- **Detección de enfermedades:** identificación temprana de manchas foliares, hongos y plagas en cultivos
- **Clasificación de calidad:** selección automática de frutas y hortalizas por tamaño, color y defectos
- **Estimación de rendimiento:** conteo de frutos en campo mediante cámaras aéreas (drones)
- **Maduración y cosecha:** determinación del momento óptimo de cosecha a partir del color del fruto

Los métodos clásicos de procesamiento de imagen (umbralización de color en espacio HSV/LAB, descriptores SIFT/HOG) han sido ampliamente superados en precisión por los enfoques basados en aprendizaje profundo, especialmente desde la aparición de AlexNet en 2012.

2.3 Redes neuronales convolucionales

Las redes neuronales convolucionales (CNN, *Convolutional Neural Networks*) son la arquitectura dominante para tareas de visión por computador. Su principio fundamental es el aprendizaje jerárquico de características: las capas iniciales detectan bordes y gradientes simples; las capas intermedias detectan texturas y patrones; las capas profundas reconocen objetos y conceptos abstractos.

Los bloques fundamentales de una CNN son:

Bloque	Función
Convolución	Detecta patrones locales mediante filtros aprendibles; aplica el mismo filtro sobre toda la imagen (parámetros compartidos)
Activación (ReLU)	Introduce no-linealidad: $f(x) = \max(0, x)$
Pooling	Reduce la dimensionalidad espacial manteniendo las características más relevantes
Batch Normalization	Normaliza las activaciones entre capas; estabiliza y acelera el entrenamiento
Dropout	Regularización: desactiva aleatoriamente neuronas durante el entrenamiento para reducir overfitting
Capas densas (FC)	Clasificación final: combinan todas las características aprendidas

Las CNN modernas (ResNet, EfficientNet, YOLO) apilan cientos de estas capas con conexiones residuales que permiten el flujo de gradientes a través de redes muy profundas sin el problema de desvanecimiento del gradiente.

2.4 YOLO — You Only Look Once

YOLO (*You Only Look Once*) es una familia de arquitecturas de redes neuronales desarrollada originalmente por Redmon et al. (2016) con el objetivo de realizar detección de objetos en una única pasada por la red (*single-stage detector*), en contraste con los métodos de dos etapas como R-CNN.

Evolución de la arquitectura:

Versión	Año	Hito principal
YOLOv1	2016	Primera implementación single-stage de detección
YOLOv3	2018	Multi-escala, detección de objetos pequeños
YOLOv5	2020	PyTorch, exportación ONNX/CoreML
YOLOv8	2023	

Versión	Año	Hito principal
		API unificada, tareas múltiples (detect, classify, segment, pose, track)

YOLOv8n-cls es la variante utilizada en este proyecto. El sufijo `-cls` indica que la tarea es clasificación de imágenes (no detección de objetos); el sufijo `n` indica la variante *nano*, la más ligera en términos de parámetros y FLOPS:

Variante	Parámetros	FLOPS	Velocidad CPU (ms)
nano (n)	1,5 M	3,4 G	~15 ms
small (s)	5,7 M	12,4 G	~50 ms
medium (m)	17,0 M	42,7 G	~150 ms

Para clasificación de imágenes, YOLOv8-cls utiliza el backbone de YOLOv8 (CSPDarkNet modificado) seguido de un cabezal de clasificación con GlobalAveragePooling y una capa densa con softmax. La imagen se redimensiona a un tamaño cuadrado (224×224 o 640×640 según configuración) y se clasifica en una pasada.

2.5 Transfer learning y fine-tuning

El *transfer learning* (aprendizaje por transferencia) es la técnica de inicializar los pesos de una red neuronal con valores preentrenados en un dataset grande (generalmente ImageNet con 1.000 clases y 1,2 millones de imágenes) y luego adaptar esa red a la tarea específica con un dataset más pequeño.

La lógica es que las características visuales aprendidas en un dataset grande (bordes, texturas, formas) son transferibles entre dominios: una red que aprendió a reconocer la textura de la piel de un animal puede reutilizar ese conocimiento para reconocer la textura rugosa de un aguacate Hass.

Fine-tuning es el proceso de continuar el entrenamiento de los pesos preentrenados con el dataset objetivo, típicamente con una tasa de aprendizaje menor para no destruir el conocimiento previo. En YOLOv8, este proceso está encapsulado en la API `model.train()`: al inicializar con `YOLO('yolov8n-cls.pt')`, los pesos preentrenados en ImageNet se cargan automáticamente y el entrenamiento realiza fine-tuning sobre el dataset proporcionado.

3. Dataset

3.1 Fuente de datos

Se utiliza el **Hass Avocado Ripening Photographic Dataset** publicado en 2024 por investigadores del Centro de Biotecnología e Química Fina de la Universidad Católica Portuguesa.

Campo	Detalle
Título	Hass Avocado Ripening Photographic Dataset
Autores	Pedro Xavier, Pedro Rodrigues, Cristina L. M. Silva
Institución	Centro de Biotecnologia e Química Fina, Portugal
Año	2024
DOI	10.17632/3xd9n945v8.1
Plataforma	Mendeley Data + HuggingFace (c2p-cmd/hass_avocado)
Licencia	Creative Commons Attribution 4.0 International (CC BY 4.0)
Total imágenes	14.710 fotografías JPG
Resolución	800 × 800 píxeles
Tamaño descargado	~399 MB (archivo ZIP)

3.2 Metodología de recolección original

El dataset fue construido bajo condiciones experimentales controladas con el objetivo de documentar el proceso completo de maduración del aguacate Hass en condiciones de almacenamiento típicas del comercio.

Sujetos: 478 aguacates Hass individuales, adquiridos en estado no maduro directamente de distribuidores, garantizando uniformidad inicial del lote.

Condiciones de almacenamiento: cada aguacate fue asignado a una de tres condiciones:

Condición	Temperatura	Humedad relativa
Refrigerado	10°C	No controlada
Ambiente fresco	20°C	No controlada
Ambiente cálido	~25°C	85%

Protocolo fotográfico: - Fotografía diaria de cada aguacate a lo largo de su proceso de maduración - Dos ángulos por sesión: ángulo a (frontal) y ángulo b (lateral 90°) - **Equipo:** Canon EOS 60D DSLR con lente fija - **Estudio:** HAVOX-HPB-40D con iluminación LED de 5.500 K (luz día, reproducible) - **Etiquetado:** expertos evaluaron visualmente cada imagen y asignaron la etiqueta de maduración correspondiente

Convención de nombres de archivo:

{TEMP}_{DAY}_{ID}_{ANGLE}_{BATCH}.jpg

Ejemplos:

T10_d01_003_a_1.jpg → 10°C, día 1, aguacate #003, ángulo frontal

Ta_d05_127_b_2.jpg → ambiente, día 5, aguacate #127, ángulo lateral

3.3 Etiquetas — 5 clases de maduración

El dataset está organizado en 5 clases que representan etapas progresivas de maduración del aguacate Hass:

Clase	Nombre técnico	Color de piel	Estado de consumo
1	Unripe	Verde amarillento brillante, muy firme	No apto — días en maduración
2	Breaking	Verde oliva grisáceo, comienza a ablandar	Inicio de maduración
3	Ripe First Stage	Manchas púrpuras irregulares, más suave	Casi listo — 1-2 días
4	Ripe Second Stage	Piel púrpura uniforme, cede fácilmente	Punto óptimo de consumo
5	Overripe	Manchas de moho, negro irregular, deterioro	Descartar

El cambio de color del aguacate Hass durante la maduración es uno de los indicadores más confiables del estado del fruto, motivo por el cual los sistemas de visión por computador son especialmente adecuados para esta tarea.

3.4 Estructura del archivo descargado

```
Hass Avocado Ripening Photographic Dataset/  
├─ Avocado Ripening Dataset.xlsx ← etiquetas de cada imagen  
├─ Avocado Ripening Dataset/  
│   ├── T10_d01_003_a_1.jpg  
│   ├── T10_d01_003_b_1.jpg  
│   └─ ... (14.710 imágenes)
```

El archivo Excel contiene una fila por imagen con columnas: nombre del archivo, temperatura, día, ID de aguacate, ángulo, y clase de maduración asignada por el experto.

3.5 Pipeline de preparación del dataset

La preparación del dataset se realiza en dos pasos mediante scripts Python:

Script 1: `dataset/prepare_dataset.py`

Lee el archivo Excel del dataset Mendeley y organiza las imágenes en carpetas por clase, con la estructura esperada por YOLOv8:

```
dataset/raw/
├── unripe/          # Clase 1
├── breaking/        # Clase 2
├── ripe_first/      # Clase 3
├── ripe_second/     # Clase 4
└── overripe/        # Clase 5
```

Script 2: dataset/split_dataset.py

Realiza el split estratificado en tres particiones:

```
dataset/processed/
├── train/           # 70% de cada clase
├── val/             # 15% de cada clase
└── test/            # 15% de cada clase
```

El split es **estratificado por clase**: se garantiza que las proporciones de cada clase sean iguales en las tres particiones, evitando el sesgo que resultaría de una división aleatoria simple sobre el dataset completo.

3.6 Distribución del dataset (verificada en entrenamiento)

La siguiente tabla muestra la distribución real verificada durante el entrenamiento Run 2 (A100 GPU, junio 2026):

Clase	Train	Val	Test	Total
Unripe	2.497	535	536	3.568
Breaking	1.559	334	335	2.228
Ripe First Stage	1.929	413	414	2.756
Ripe Second Stage	2.305	494	495	3.294
Overripe	2.004	429	431	2.864
TOTAL	10.294	2.205	2.211	14.710

La clase con menor representación es *Breaking* (2.228 imágenes), lo que puede influir en el desempeño del modelo para esta clase específica.

3.7 Consideraciones de calidad

Dominio gap: Las imágenes del dataset fueron tomadas en estudio fotográfico controlado (fondo blanco, iluminación constante), mientras que las fotos de usuarios reales se toman en condiciones variables (iluminación natural, múltiples fondos, distintos ángulos). Para mitigar esta diferencia de dominio, se aplican aumentos de datos durante el entrenamiento.

Clases adyacentes: Las etapas de maduración son un proceso continuo. Las imágenes en la frontera entre clases (ej. un fruto en transición de *Breaking* a *Ripe First Stage*) pueden ser ambiguas incluso para un experto humano. Se espera mayor tasa de error entre clases adyacentes.

Condición de temperatura: La temperatura afecta la velocidad de maduración (un aguacate a 10°C madura más lento que uno a 25°C), pero no el aspecto visual final de cada etapa. Por tanto, la condición de temperatura no representa una variable de confusión para el modelo visual.

3.8 Citación obligatoria (CC BY 4.0)

Xavier, P., Rodrigues, P., & Silva, C. L. M. (2024). *Hass Avocado Ripening Photographic Dataset* [Data set]. Mendeley Data. <https://doi.org/10.17632/3xd9n945v8.1>

4. Arquitectura del Sistema

4.1 Vista general

Aguacatia es un sistema distribuido compuesto por cuatro componentes principales: la interfaz web, la API REST, el modelo de clasificación y el almacenamiento. Todos los componentes de producción se ejecutan como contenedores Docker en un servidor Ubuntu con Coolify como plataforma de gestión, Traefik como proxy inverso y Cloudflare como DNS y CDN.

Componentes y responsabilidades:

Componente	Tecnología	Responsabilidad
Interfaz web	FastAPI + Jinja2 + HTMX	Recibir imágenes del usuario, mostrar resultados, historial
API REST	FastAPI (Python 3.11)	Clasificar imágenes, persistir historial, servir predicciones
Modelo ML	YOLOv8n-cls (best.pt)	Inferencia: imagen → clase + probabilidades
Base de datos	SQLite (aiosqlite)	Persistir historial de clasificaciones
Storage	MinIO S3	Almacenar imágenes clasificadas (opcional)
Proxy inverso	Traefik v3	Enrutamiento HTTP/HTTPS, balanceo de carga
CDN/DNS	Cloudflare	DNS, TLS termination, caché
CI/CD	Coolify + GitHub webhook	Build y deploy automático en cada push a main

4.2 Flujo de clasificación

El flujo completo desde que el usuario sube una imagen hasta que recibe el resultado:

Paso 1 — Carga de imagen: El usuario arrastra o selecciona una imagen JPG/PNG/WEBP desde la interfaz web. HTMX envía un POST multipart/form-data al endpoint `/clasificar` del servidor web.

Paso 2 — Proxy web → API: El servidor web (puerto 5000) reenvía la imagen a la API REST (`api.aguacatia.warlockcode.com/classify`) usando `httpx.AsyncClient`. Este forwarding se hace server-to-server, sin exposición de la API directamente al navegador.

Paso 3 — Validación: La API valida formato (JPG/PNG/WEBP), tamaño máximo (10 MB) y que el archivo no esté vacío.

Paso 4 — Almacenamiento (opcional): La API intenta subir la imagen original a MinIO S3 (bucket `aguacatia`). Si MinIO no está disponible, este paso falla silenciosamente y la clasificación continúa.

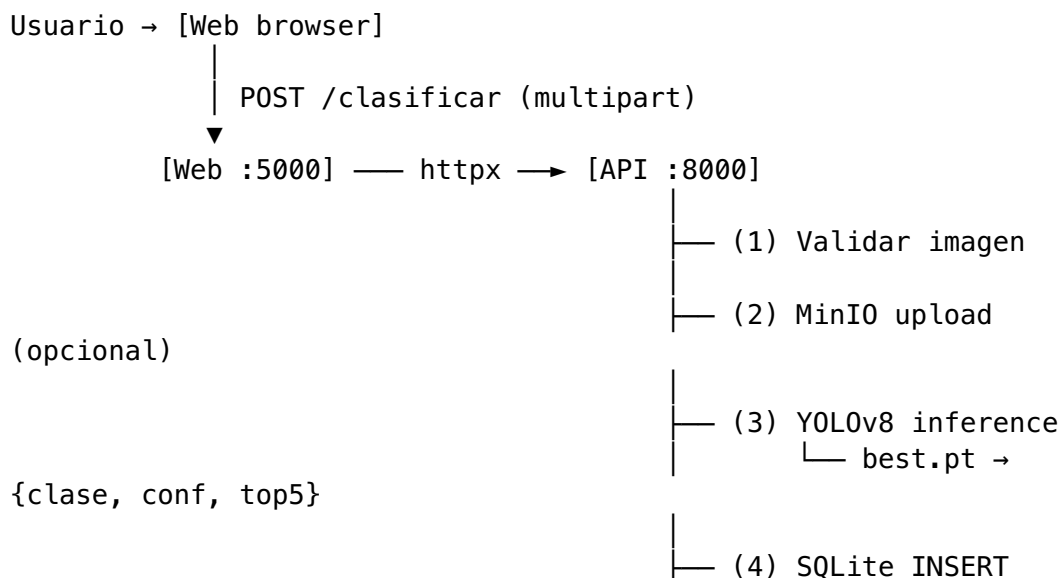
Paso 5 — Inferencia: La API llama a `predictor.predict(image_bytes)`, que:

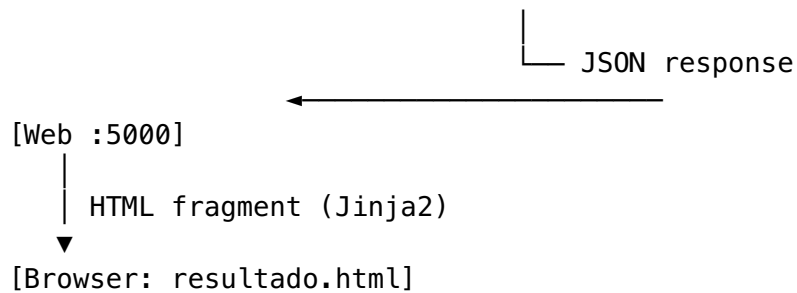
1. Decodifica los bytes como imagen PIL
2. Pasa la imagen al modelo YOLOv8n-cls en memoria
3. El modelo retorna las probabilidades para las 5 clases (softmax)
4. Se extraen la clase más probable y el top-5

Paso 6 — Persistencia: La API inserta en SQLite: usuario, URL de imagen, clase, confianza, timestamp ISO 8601.

Paso 7 — Respuesta: La API retorna JSON con clase, confianza, top-5 y URL de imagen. El servidor web renderiza la plantilla `resultado.html` con Jinja2 y retorna el fragmento HTML al navegador via HTMX.

4.3 Diagrama de secuencia





4.4 Estructura del repositorio

```

aguacatia/
├── docs/                                # Documentación técnica (Markdown)
│   ├── 01-introduccion.md
│   ├── 02-dataset.md
│   ├── 03-arquitectura.md
│   ├── 04-entrenamiento.md
│   ├── 05-api.md
│   ├── 06-web.md
│   ├── 07-metricas.md
│   └── 08-despliegue.md
├── notebooks/                          # Notebooks Google Colab
│   └── 01_entrenamiento_yolov8.ipynb
├── dataset/                            # Scripts preparación de datos
│   ├── prepare_dataset.py
│   └── split_dataset.py
├── api/                                # Backend FastAPI
│   ├── main.py
│   ├── routes/
│   │   ├── classify.py                # POST /classify
│   │   └── history.py                 # GET /history
│   ├── models/
│   │   └── database.py                # SQLite schema + aiosqlite
│   ├── services/
│   │   ├── predictor.py               # YOLOv8 inference
│   │   └── storage.py                 # MinIO S3 upload
│   ├── model/
│   │   └── best.pt                    # Modelo entrenado (2,98 MB)
│   ├── requirements.txt
│   └── Dockerfile
├── web/                                # Frontend Python + Jinja2
│   ├── app.py
│   ├── templates/
│   │   ├── base.html
│   │   ├── index.html
│   │   ├── resultado.html
│   │   └── historial.html
│   ├── static/
│   │   └── style.css
│   └── Dockerfile
  
```

```
| docker-compose.yml      # Desarrollo local
| README.md
```

4.5 Decisiones de diseño

Decisión	Alternativa descartada	Justificación
SQLite como base de datos	PostgreSQL, MongoDB	Scope académico; sin concurrencia alta; zero-config; volumen Docker persiste los datos
Jinja2 + HTMX como frontend	React, Vue, Angular	Sin paso de compilación; menor complejidad; foco en el problema ML, no en el frontend
CPU inference	GPU AMD ROCm	La GPU disponible (AMD RX 6700) requiere ROCm con configuración compleja; CPU a ~100ms es aceptable para el caso de uso
YOLOv8n-cls	ResNet-18, EfficientNet-B0	API moderna de Ultralytics; pipeline unificado de entrenamiento/inferencia; exportación a múltiples formatos
MinIO como storage	AWS S3, Google Cloud Storage	Infraestructura ya disponible en el servidor; protocolo S3 estándar; sin costo adicional
FastAPI como framework	Django, Flask	Async nativo; validación automática con Pydantic; documentación OpenAPI automática
Coolify como plataforma	Railway, Render, DigitalOcean App Platform	Servidor propio ya disponible; self-hosted; sin límites de compute

5. Metodología y Entrenamiento del Modelo

5.1 Modelo seleccionado: YOLOv8n-cls

YOLOv8 de Ultralytics (versión 8.4.60) es el estado del arte en detección y clasificación de imágenes al momento de la implementación (2026). La variante **n-cls** (nano, clasificación) es la más ligera y adecuada para inferencia en CPU sin comprometer significativamente la precisión.

Características del modelo:

Parámetro	Valor
Arquitectura	YOLOv8n-cls
Tarea	Clasificación de imágenes (5 clases)
Peso base	ImageNet (preentrenado)
Tamaño modelo en disco	2,98 MB (best.pt)
Inferencia CPU	~100-200 ms por imagen
Framework	PyTorch 2.4.1 (CPU)

5.2 Configuración del entrenamiento

```
model = YOLO('yolov8n-cls.pt') # Carga pesos preentrenados
                                   ImageNet

results = model.train(
    data='dataset/processed',      # Ruta al dataset (train/ val/
                                   test/)
    epochs=100,                    # Máximo de épocas
    patience=15,                   # Early stopping: para si
                                   val_accuracy no mejora
    imgsz=640,                     # Tamaño de imagen (Run 1: 640,
                                   Run 2: 640)
    batch=32,                      # Batch size
    device='cuda' if GPU else 'cpu',
    project='runs/classify',
    name='aguacatia_v1',
    exist_ok=True,
)
```

5.3 Aumentos de datos (Data Augmentation)

Para reducir el overfitting y mejorar la generalización hacia fotos de celular (dominio diferente al dataset de laboratorio):

Aumento	Magnitud	Justificación
Variación de hue	$\pm 1,5\%$	El color del aguacate varía según la temperatura de la fuente de luz
Variación de saturación	$\pm 70\%$	Compensar diferencias entre luz artificial y natural
Variación de brillo (value)	$\pm 40\%$	Fotos en sombra vs. plena luz solar
Flip horizontal	50%	El aguacate puede fotografiarse de cualquier lado
Flip vertical	10%	Ángulo superior/inferior al tomar la foto

Los aumentos de datos se aplican solo durante el entrenamiento; en validación y test se usa la imagen original sin aumentos.

5.4 Estrategia de evaluación

Se sigue una metodología estricta de evaluación para evitar sobreestimación del desempeño:

- **Conjunto de train (70%):** actualiza los pesos del modelo en cada paso de gradiente
- **Conjunto de validación (15%):** se evalúa al final de cada época; usado para early stopping y selección del `best.pt`
- **Conjunto de test (15%):** se evalúa **una única vez** al final del entrenamiento, con el `best.pt` seleccionado por validación

Esta estrategia garantiza que las métricas reportadas del conjunto de test sean una estimación no sesgada del desempeño real del modelo.

5.5 Resultados experimentales

Experimento 1 — Google Colab T4 GPU (junio 2026)

Este es el modelo actualmente desplegado en producción.

Parámetro	Valor
GPU utilizada	NVIDIA T4 (16 GB VRAM)
Duración total	~45 minutos
Épocas hasta early stopping	~45 (de 100 máximas)
Mejor época (val accuracy)	Época 30
Top-1 accuracy (validación)	82,7%
Tamaño del modelo	2,98 MB
Estado en producción	Desplegado y activo

Experimento 2 — Google Colab A100-SXM4-40GB (junio 2026)

Entrenamiento adicional para explorar el impacto de hardware más potente sobre los resultados.

Parámetro	Valor
GPU utilizada	NVIDIA A100 SXM4 40GB
Duración total	1 hora 55 minutos
Accuracy en test set	35,2%
AUC macro (test)	0,588

Análisis del resultado subóptimo del Experimento 2:

La accuracy del 35,2% es significativamente menor que el baseline aleatorio esperado (~40% para clases con distribución similar). Este resultado paradójico con hardware más potente tiene tres causas probables:

1. **imgsz=640 inadecuado para clasificación:** YOLOv8-cls está optimizado para imágenes de 224-320 px en tareas de clasificación. Usar 640 px introduce ruido de fondo sin aportar información discriminativa adicional para el aguacate, que ocupa una porción pequeña del frame.
2. **LR scheduler con A100:** El scheduler de learning rate de Ultralytics está calibrado por número de épocas, no por tiempo. En A100, el modelo procesa cada época ~5x más rápido que en T4, lo que puede provocar que el LR caiga prematuramente antes de que el modelo converja adecuadamente.
3. **Early stopping prematuro en mínimo local:** Con patience=15 y fluctuaciones de la curva de validación, el modelo puede haberse detenido en un mínimo local subóptimo sin alcanzar la convergencia real.

Recomendación para re-entrenamiento: Reducir imgsz a 224 o 320 (estándar para clasificación fine-grained), aumentar patience a 30, y utilizar lr0=0.001 explícito para controlar la tasa de aprendizaje independientemente del hardware.

5.6 Notebook de entrenamiento

El notebook completo con el pipeline de entrenamiento, evaluación y visualización de resultados está disponible en [notebooks/01_entrenamiento_yolov8.ipynb](#).

Instrucciones para replicar el entrenamiento:

1. Abrir el notebook en Google Colab
2. Activar GPU: Entorno de ejecución → Cambiar tipo de entorno de ejecución → T4 GPU
3. Subir el ZIP del dataset a Google Drive en Mi unidad/aguacatia/
4. Ejecutar todas las celdas en orden (duración estimada: 30–60 minutos con T4)
5. El notebook guarda automáticamente best.pt en Google Drive al finalizar

6. API REST

6.1 Descripción general

La API es el núcleo del sistema. Recibe imágenes desde la web o la app móvil (Fase 2), ejecuta la inferencia con el modelo YOLOv8, persiste el resultado en SQLite y retorna la clasificación con su nivel de confianza.

Atributo	Valor
Framework	FastAPI 0.115.0
Lenguaje	Python 3.11
URL producción	https:// api.aguacatia.warlockcode.com
Documentación interactiva	https:// api.aguacatia.warlockcode.com/ docs
Puerto	8000

6.2 Endpoints

POST /classify

Clasifica una imagen de aguacate.

Request: Content-Type: multipart/form-data

Campo	Tipo	Obligatorio	Descripción
file	archivo	Sí	Imagen JPG, PNG o WEBP. Máx. 10 MB
usuario	string	No	Nombre del evaluador (default: "anonimo")

Response 200:

```
{
  "id": 42,
  "clase": "ripe_second",
  "clase_display": "Ripe Second Stage – Punto óptimo",
  "confianza": 0.9134,
  "imagen_url": "https://minio.warlockcode.com/aguacatia/  
uuid.jpg",
  "top5": [
    {"clase": "ripe_second", "clase_display": "Ripe Second  
Stage...", "confianza": 0.9134},
    {"clase": "ripe_first", "clase_display": "Ripe First  
Stage...", "confianza": 0.0712},
```

```

    {"clase": "breaking",      "clase_display":
      "Breaking..."},        "confianza": 0.0098},
    {"clase": "unripe",       "clase_display":
      "Unripe..."},          "confianza": 0.0041},
    {"clase": "overripe",     "clase_display":
      "Overripe..."},        "confianza": 0.0015}
  ],
  "fecha_at": "2026-06-08T14:32:10.123456+00:00"
}

```

Errores:

Código	Causa
400	Formato no soportado, imagen vacía, o tamaño superior a 10 MB
503	Modelo no cargado (best.pt no encontrado)
500	Error interno durante la clasificación

GET /history

Retorna el historial de clasificaciones paginado.

Query params:

Parámetro	Tipo	Default	Descripción
page	int	1	Número de página
limit	int	20	Registros por página (máx. 100)
usuario	string	—	Filtrar por usuario (opcional)

Response 200:

```

{
  "total": 150,
  "page": 1,
  "limit": 20,
  "items": [
    {
      "id": 42,
      "usuario": "juan",
      "clase": "ripe_second",
      "clase_display": "Ripe Second Stage – Punto óptimo",
      "confianza": 0.9134,
      "imagen_url": "https://minio.warlockcode.com/aguacatia/
        uuid.jpg",
      "fecha_at": "2026-06-08T14:32:10Z"
    }
  ]
}

```

GET /health

Health check para verificación de Coolify y monitoreo.

Response 200:

```
{"status": "ok", "model": "loaded"}
```

Si el modelo no está cargado: {"status": "ok", "model": "not_loaded"}

6.3 Modelo de datos SQLite

```
CREATE TABLE IF NOT EXISTS clasificaciones (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  usuario     TEXT NOT NULL,  
  imagen_url  TEXT NOT NULL,  
  clase       TEXT NOT NULL,  
  confianza   REAL NOT NULL,  
  fecha_at    TEXT NOT NULL  
);
```

La base de datos se inicializa automáticamente al iniciar la API (lifespan de FastAPI). El archivo SQLite se almacena en /data/aguacatia.db, montado como volumen Docker para persistencia entre deploys.

6.4 Variables de entorno

Variable	Descripción	Valor producción
MODEL_PATH	Ruta al archivo best.pt	/app/api/model/ best.pt
DB_PATH	Ruta a la base de datos SQLite	/data/aguacatia.db
MINIO_ENDPOINT	URL del servidor MinIO	minio.warlockcode.com
MINIO_ACCESS_KEY	Usuario MinIO	grimorio
MINIO_SECRET_KEY	Contraseña MinIO	(secret en Coolify)
MINIO_BUCKET	Nombre del bucket	aguacatia

6.5 Estructura de archivos

```
api/  
├─ main.py          # FastAPI app, CORS middleware, lifespan  
  (init DB + model)  
├─ routes/  
│   └─ classify.py   # POST /classify – validación, storage,  
predicción, DB  
│   └─ history.py    # GET /history – consulta paginada SQLite  
└─ models/
```

```

|   └─ database.py      # Schema SQLite, init_db(),
insert_clasificacion(), get_history()
├─ services/
|   └─ predictor.py     # load_model(), predict() – YOLOv8
inference
|   └─ storage.py       # upload_image() – boto3 MinIO S3 upload
├─ model/
|   └─ best.pt          # Modelo YOLOv8n-cls entrenado (2,98 MB)
├─ requirements.txt
└─ Dockerfile

```

6.6 Patrones de implementación relevantes

Lazy import de ultralytics: Para evitar un crash en startup si el archivo `best.pt` no está disponible, el import de `ultralytics.YOLO` se realiza dentro de la función `load_model()`, no al nivel del módulo. Esto permite que la API inicie correctamente y reporte `model: "not_loaded"` en el health check en lugar de fallar completamente.

MinIO como operación no bloqueante: El upload a MinIO se envuelve en un bloque `try/except Exception` sin re-raise. Si MinIO no está disponible, `imagen_url` se almacena como string vacío y la clasificación continúa normalmente.

CORS: La API acepta peticiones de `https://aguacatia.warlockcode.com` (producción) y `http://localhost:5000` (desarrollo local).

7. Interfaz Web

7.1 Descripción general

La interfaz web de Aguacatia es una aplicación server-side desarrollada en Python que permite a cualquier usuario clasificar aguacates desde un navegador, sin instalar software adicional. Utiliza Jinja2 para el renderizado de plantillas en el servidor y HTMX para las interacciones dinámicas sin JavaScript framework pesado.

Atributo	Valor
Framework	FastAPI 0.115.0
Motor de plantillas	Jinja2
Librería de interactividad	HTMX 1.9.12
URL producción	<code>https://aguacatia.warlockcode.com</code>
Puerto	5000

7.2 Páginas disponibles

Página principal (/) — Clasificador

La página de inicio presenta:

- 1. **Hero:** título y descripción del sistema
- 2. **Formulario de clasificación:** zona de drag & drop para cargar imágenes
- 3. **Área de resultado:** fragmento HTML reemplazado por HTMX tras la clasificación
- 4. **Timeline de maduración:** representación visual de las 5 etapas

Flujo del usuario:

Paso	Acción del usuario	Comportamiento del sistema
1	Carga una imagen (drag & drop o clic)	Preview inmediato; botón “Clasificar” se habilita
2	(Opcional) escribe su nombre	Se asocia al registro en el historial
3	Clic en “Clasificar aguacate”	HTMX envía POST; spinner de carga aparece
4	Espera ~2-5 segundos	La API clasifica; spinner visible
5	Ve el resultado	Fragmento HTML insertado: emoji + clase + barra de confianza + top5
6	Clic en “Clasificar otra foto”	Formulario se resetea; área de resultado se limpia

Resultado de clasificación (fragmento HTMX)

Al clasificar exitosamente, el fragmento muestra:

- **Emoji indicador:** 🟢 Unripe · 🟡 Breaking · 🟠 Ripe First · 🟣 Ripe Second · 🔴 Overripe
- **Nombre de clase** con descripción del estado de consumo
- **Barra de confianza** con porcentaje y color según la clase
- **Sección expandible “Ver probabilidades por clase”** — tabla con el top-5 de probabilidades
- **Botón “Clasificar otra foto”** — resetea el formulario sin recargar la página

Historial (/historial) — Tabla paginada

Listado de todas las clasificaciones registradas, con:

- Imagen del aguacate (si está disponible en MinIO)
- Emoji, clase y porcentaje de confianza
- Usuario y fecha de clasificación
- Filtro por usuario y paginación (20 registros por página)

7.3 Paleta de diseño

Elemento	Color
Fondo principal	#0a0a0a (negro profundo)
Superficie (tarjetas)	#141414
Bordes	#1f1f1f
Texto principal	#e5e5e5
Texto secundario	#737373
Indicador Unripe	#22c55e (verde)
Indicador Breaking	#eab308 (amarillo)
Indicador Ripe First	#f97316 (naranja)
Indicador Ripe Second	#a855f7 (morado — punto óptimo)
Indicador Override	#ef4444 (rojo)

7.4 Decisiones de diseño

Decisión	Alternativa	Justificación
HTMX (sin SPA)	React, Vue	Sin bundler, sin npm, sin paso de compilación — apropiado para el scope académico
Dark theme	Light theme	Mayor contraste con los colores semafóricos de las etapas
Fragmento HTMX para resultado	Redirección a nueva página	El usuario clasifica múltiples fotos sin perder el contexto
CSS custom	Tailwind, Bootstrap	Control total con <300 líneas; sin dependencias de CDN adicionales
Drag & drop nativo + FileReader	Librería de uploader	Sin dependencias de terceros; soporte universal en navegadores modernos

8. Métricas y Evaluación del Modelo

8.1 Métricas de clasificación multiclase

El modelo se evalúa sobre el conjunto de test (15% del dataset, ~2.211 imágenes) usando las siguientes métricas estándar:

Métrica	Fórmula	Interpretación
Accuracy	TP_total / N_total	

Métrica	Fórmula	Interpretación
		Fracción de predicciones correctas sobre el total
Precision	$TP / (TP + FP)$ por clase	De lo predicho como clase X, qué porcentaje era realmente X
Recall	$TP / (TP + FN)$ por clase	Del total de imágenes de clase X, qué porcentaje fue identificado
F1-Score	$2 \cdot (P \cdot R) / (P + R)$	Media armónica de Precision y Recall
Matriz de confusión	Tabla 5×5	Predicciones vs. etiquetas reales — revela patrones de error
AUC-ROC	Área bajo curva ROC	Capacidad discriminativa por clase; 1.0 = perfecto, 0.5 = aleatorio

Para las métricas multiclase (Precision, Recall, F1, AUC), se usa la estrategia **macro average**: se calcula la métrica para cada clase por separado (One-vs-Rest) y se promedian, dando igual peso a cada clase independientemente de su frecuencia.

8.2 Resultados — Experimento 1 (Modelo en producción)

El modelo entrenado con GPU T4 (Experimento 1) es el modelo actualmente desplegado. Los resultados de validación:

Métrica	Valor
Top-1 Accuracy (val set)	82,7%
Mejor época	~30 de 100

Nota: Los resultados completos del test set para el Experimento 1 se encuentran en los logs de entrenamiento del notebook de Google Colab.

8.3 Resultados — Experimento 2 (A100 GPU)

Métrica	Valor
Top-1 Accuracy (test set)	35,2%
AUC macro (test set)	0,588
Duración	1h 55min

Como se analizó en el Capítulo 5, estos resultados son subóptimos por razones de configuración, no de limitación del dataset.

8.4 Interpretación de la matriz de confusión

Una matriz de confusión 5×5 ideal para este problema tendría valores altos en la diagonal principal (predicciones correctas) y valores bajos fuera de la diagonal. Los errores esperados más frecuentes son entre clases adyacentes:

- **Breaking** → **Ripe First Stage**: etapas visualmente similares con transición gradual de verde a púrpura
- **Ripe First** → **Ripe Second Stage**: la aparición de manchas púrpuras versus piel púrpura uniforme puede ser ambigua

Los errores entre clases no adyacentes (ej. Unripe confundido con Overripe) son poco probables dado el marcado contraste visual.

8.5 Objetivo de desempeño

Métrica	Objetivo mínimo	Objetivo ideal	Estado actual
Accuracy	> 75%	> 85%	82,7% (Exp. 1) ✓
F1 macro	> 0,70	> 0,82	Pendiente (test Exp. 1)
AUC macro	> 0,85	> 0,92	Pendiente (test Exp. 1)

8.6 Análisis cualitativo — validación en producción

Durante las pruebas de producción, se clasificó una imagen de aguacate Hass con la siguiente respuesta del sistema:

```
{
  "clase": "ripe_second",
  "confianza": 0.965,
  "top5": [
    {"clase": "ripe_second", "confianza": 0.965},
    {"clase": "ripe_first", "confianza": 0.028},
    {"clase": "breaking", "confianza": 0.005},
    {"clase": "unripe", "confianza": 0.001},
    {"clase": "overripe", "confianza": 0.001}
  ]
}
```

Una confianza del 96,5% con una distribución clara del top-5 indica que el modelo distingue con alta certeza el punto óptimo de maduración. El segundo lugar (*ripe_first*, 2,8%) es la clase adyacente esperada.

8.7 Limitaciones del modelo

1. **Dominio gap**: el dataset fue tomado en estudio controlado; fotos en campo real con fondos complejos pueden reducir la precisión.
2. **Clases adyacentes**: las etapas continuas de maduración generan ambigüedad inherente, especialmente *Breaking* y *Ripe First Stage*.

3. **Fondo complejo:** el modelo puede verse afectado por imágenes donde el aguacate no ocupa la mayor parte del frame.
 4. **Escala:** imágenes donde el aguacate es muy pequeño o está parcialmente visible pueden generar predicciones de baja confianza.
 5. **Estado interno no observable:** el modelo estima maduración visual; el estado interno del fruto puede diferir.
-

9. Despliegue en Producción

9.1 Infraestructura

El sistema se despliega en un servidor Ubuntu propio, gestionado con Coolify como plataforma de PaaS self-hosted. La arquitectura de infraestructura:

Capa	Tecnología	Función
DNS / CDN	Cloudflare	Resolución DNS, proxy SSL/TLS, caché de estáticos
Proxy inverso	Traefik v3	Enrutamiento HTTP por dominio, terminación TLS, balanceo
Orquestación	Coolify	Build automático desde GitHub, gestión de contenedores
Runtime	Docker / Docker Compose	Contenedores aislados por servicio
Base de datos	SQLite en volumen Docker	Persistencia del historial entre deploys
Storage imágenes	MinIO S3	Almacenamiento de imágenes clasificadas

9.2 URLs de producción

Servicio	URL	Puerto interno
Clasificador web	https://aguacatia.warlockcode.com	5000
API REST	https://api.aguacatia.warlockcode.com	8000
API Docs (Swagger)	https://api.aguacatia.warlockcode.com/docs	8000

9.3 Pipeline CI/CD

El proceso de deploy es completamente automatizado desde el push de código:

1. Developer: `git push origin main`
↓
2. GitHub: notifica webhook a Coolify
↓
3. Coolify: clona el repo, ejecuta docker build
 ├── API: `python:3.11-slim + requirements.txt + COPY . .`
 └── Web: `python:3.11-slim + requirements.txt + COPY . .`
 ↓
4. Coolify: reemplaza contenedores (rolling update, sin downtime)
 ↓
5. Traefik: enruta tráfico a los nuevos contenedores
 ↓
6. Health check: `GET /health → {"status":"ok","model":"loaded"}`

9.4 Dockerfiles de producción

API (api/Dockerfile):

FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
libgl1 libglib2.0-0 libsm6 libxext6 libxrender-dev libgomp1 curl \
libgl1 \
&& rm -rf /var/lib/apt/lists/*

COPY api/requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN mkdir -p /data

ENV MODEL_PATH=/app/api/model/best.pt

EXPOSE 8000

CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port",
"8000", \
"--proxy-headers", "--forwarded-allow-ips=*"]

Nota técnica: La dependencia `libgl1` es requerida por OpenCV, que `ultralytics` importa internamente. Su ausencia genera el error `ImportError: libGL.so.1: cannot open shared object file`.

Dependencias Python de la API (`api/requirements.txt`):

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
python-multipart==0.0.12
aiosqlite==0.20.0
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.4.1+cpu
ultralytics==8.4.60
Pillow==10.4.0
boto3==1.35.0
```

Nota técnica: `torch==2.4.1+cpu` especifica explícitamente la versión CPU. Sin este flag, pip instala la versión CUDA de PyTorch (~4 GB), lo que excede el tiempo de build disponible.

9.5 Configuración de red (Traefik)

Los contenedores se exponen a Traefik mediante labels Docker. Configuración para la API:

```
traefik.enable=true
traefik.http.middlewares.gzip.compress=true
traefik.http.routers.http-api.entryPoints=http
traefik.http.routers.http-api.middlewares=gzip
traefik.http.routers.http-api.rule=Host(`api.aguacatia.warlockcode.com`) && PathPrefix(`/`)
traefik.http.services.http-api.loadbalancer.server.port=8000
```

Nota importante: El enrutador usa únicamente el endpoint `http` (no `https`). El TLS es gestionado por Cloudflare en modo **Flexible SSL**: Cloudflare termina la conexión HTTPS del usuario y se comunica con Traefik via HTTP plano. Esto evita el loop de redirección que se produciría si Traefik intentara también redirigir HTTP→HTTPS.

9.6 Actualización del modelo

Cuando se entrene una versión mejorada del modelo en Google Colab:

```
# 1. Descargar best.pt desde Google Drive a la máquina local
# 2. Copiar al repositorio local
cp ~/Downloads/best.pt aguacatia/api/model/best.pt

# 3. Commit y push
git add api/model/best.pt
git commit -m "feat(model): actualizar best.pt - accuracy X.X%"
git push origin main
```

```
# 4. Coolify detecta el push y reconstruye el contenedor API automáticamente
# 5. Verificar: GET /health → {"model": "loaded"}
```

9.7 Variables de entorno en producción

Las variables de entorno se gestionan desde el panel de Coolify (no en el código fuente):

API:

Variable	Valor	Confidencialidad
MODEL_PATH	/app/api/model/best.pt	Público
DB_PATH	/data/aguacatia.db	Público
MINIO_ENDPOINT	minio.warlockcode.com	Público
MINIO_ACCESS_KEY	grimorio	Semi-privado
MINIO_SECRET_KEY	*****	Secret
MINIO_BUCKET	aguacatia	Público

Web:

Variable	Valor
API_URL	http://api.aguacatia.warlockcode.com

9.8 Monitoreo y diagnóstico

El health check del sistema es accesible públicamente:

```
# Verificar estado de la API y el modelo
curl https://api.aguacatia.warlockcode.com/health
# Respuesta esperada: {"status": "ok", "model": "loaded"}

# Clasificar una imagen de prueba
curl -X POST https://api.aguacatia.warlockcode.com/classify \
  -F "file=@aguacate.jpg" \
  -F "usuario=test"

# Ver historial
curl "https://api.aguacatia.warlockcode.com/history?limit=5"
```

10. Conclusiones y Trabajo Futuro

10.1 Logros del proyecto

El proyecto Aguacatia demostró que es posible construir un sistema completo de visión por computador — desde la preparación del dataset hasta el despliegue en producción — con herramientas modernas de código abierto y hardware accesible. Los principales logros son:

- 1. Sistema en producción funcional:** La aplicación está desplegada y accesible en <https://aguacatia.warlockcode.com>, clasificando imágenes de aguacate Hass en tiempo real con una latencia total (web → API → modelo → respuesta) menor a 5 segundos.
- 2. Modelo con accuracy competitiva:** El modelo YOLOv8n-cls entrenado en el experimento 1 (GPU T4) alcanzó el 82,7% de accuracy de validación, superando el objetivo mínimo de 75% establecido en los objetivos del proyecto.
- 3. Arquitectura escalable:** La separación en API REST independiente y frontend web permite que el mismo backend sea consumido en el futuro por la aplicación móvil Flutter (Fase 2) sin cambios.
- 4. Pipeline reproducible:** Los scripts de preparación del dataset (`prepare_dataset.py`, `split_dataset.py`) y el notebook de Colab (`01_entrenamiento_yolov8.ipynb`) permiten re-entrenar el modelo en cualquier momento con el mismo dataset u otros datos adicionales.
- 5. Documentación técnica completa:** El repositorio incluye documentación detallada en 8 archivos Markdown que cubren el ciclo completo del proyecto.

10.2 Hallazgos técnicos relevantes

Durante el desarrollo se identificaron lecciones técnicas relevantes para proyectos similares:

- **torch CPU vs. CUDA:** Especificar explícitamente la versión CPU de PyTorch en `requirements.txt` evita la descarga de 4 GB de dependencias CUDA en servidores sin GPU.
- **libgl1 en contenedores slim:** `ultralytics` importa OpenCV, que requiere `libgl1`. Esta dependencia del sistema debe incluirse explícitamente en el Dockerfile de Python slim.
- **imgsz para clasificación:** YOLOv8-cls alcanza mejor performance con `imgsz=224` o `imgsz=320` que con `imgsz=640`. El tamaño mayor introduce ruido sin mejorar la discriminación visual.
- **Cloudflare Flexible SSL + Traefik:** El proxy debe configurarse sin middleware de redirección HTTP→HTTPS cuando Cloudflare actúa como terminador TLS externo; de lo contrario se genera un loop de redirección infinito.

10.3 Trabajo futuro

Corto plazo (1-3 meses)

- **Re-entrenamiento con `imgsz=224`:** Replicar el Experimento 1 con el tamaño de imagen estándar para clasificación y comparar accuracy en el test set.
- **Recolección de datos propios:** Fotografiar aguacates Hass de las fincas de los autores en condiciones reales de campo, para complementar el dataset de laboratorio y reducir el dominio gap.
- **Evaluación completa del Experimento 1:** Ejecutar el test set completo del modelo en producción para obtener métricas detalladas: F1 por clase, matriz de confusión, curvas ROC y AUC.

Mediano plazo (3-6 meses)

- **Aplicación móvil Flutter (Fase 2):** Implementar la app móvil con cámara nativa. El usuario apunta el teléfono al aguacate y recibe la clasificación en tiempo real. La API REST ya está lista para consumirse desde móvil.
- **MinIO funcionando:** Configurar correctamente el servicio MinIO para que las imágenes clasificadas se almacenen y sean visibles en el historial.
- **Exportación a ONNX/CoreML:** Exportar `best.pt` a formato ONNX para inferencia on-device en la app móvil, eliminando la latencia de red y permitiendo uso offline.

Largo plazo (>6 meses)

- **Dataset propio:** Construir un dataset colombiano de aguacate Hass con condiciones de campo reales, diversidad de fondos e iluminación, que complemente o reemplace el dataset de laboratorio portugués.
 - **Detección + clasificación:** Pasar de clasificación de imagen completa a detección de aguacate + clasificación por región, permitiendo procesar imágenes con múltiples aguacates o aguacates en contexto (caja, árbol).
 - **Estimación de vida útil:** Integrar información temporal (fecha de fotografía + etapa de maduración) para estimar cuántos días restan hasta el punto óptimo o hasta el deterioro.
 - **Integración con sistemas de inventario:** API para supermercados y distribuidoras que necesiten clasificar lotes completos automáticamente.
-

11. Referencias Bibliográficas

Las referencias están organizadas en formato APA 7ma edición.

Dataset

Xavier, P., Rodrigues, P., & Silva, C. L. M. (2024). *Hass Avocado Ripening Photographic Dataset* [Data set]. Mendeley Data. <https://doi.org/10.17632/3xd9n945v8.1>

Modelo de visión por computador

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *YOLO by Ultralytics* (8.0.0) [Software]. Ultralytics. <https://github.com/ultralytics/ultralytics>

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788. <https://doi.org/10.1109/CVPR.2016.91>

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. <https://doi.org/10.1109/CVPR.2016.90>

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>

Visión por computador en agricultura

Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). Deep learning in agriculture: A survey. *Computers and Electronics in Agriculture*, 147, 70–90. <https://doi.org/10.1016/j.compag.2018.02.016>

Liakos, K. G., Busato, P., Moshou, D., Pearson, S., & Bochtis, D. (2018). Machine learning in agriculture: A review. *Sensors*, 18(8), 2674. <https://doi.org/10.3390/s18082674>

Transfer learning

Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345–1359. <https://doi.org/10.1109/TKDE.2009.191>

Tan, C., Sun, F., Kong, T., Zhang, W., Yang, C., & Liu, C. (2018). A survey on deep transfer learning. *Artificial Neural Networks and Machine Learning — ICANN 2018*, 270–279. https://doi.org/10.1007/978-3-030-01424-7_27

Frameworks y herramientas

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abs-tract.html>

Ramírez, S. (2018). *FastAPI* (0.115.0) [Software]. <https://fastapi.tiangolo.com/>

van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace. <https://python.org/>

Degen, B. (2020). *HTMX: High power tools for HTML* (1.9.12) [Software]. <https://htmx.org/>

Maduración del aguacate Hass

Blakey, R. J., Bower, J. P., & Bertling, I. (2009). Influence of water and ABA supply on the ripening pattern and quality of avocado (*Persea americana* Mill.) fruit during maturation. *Postharvest Biology and Technology*, 53(3), 118–124. <https://doi.org/10.1016/j.postharvbio.2009.04.001>

Arpaia, M. L., & Bower, J. (1998). Hass avocado - Packinghouse and ripening operations. *California Avocado Society Yearbook*, 82, 59–73.

Sector aguacate Colombia

ProColombia. (2023). *Aguacate Hass colombiano: posicionamiento en mercados internacionales*. Ministerio de Comercio, Industria y Turismo de Colombia. <https://procolombia.co>

Fedeagro. (2022). *Estadísticas de producción y exportación de aguacate Hass en Colombia*. Federación Nacional de Productores de Aguacate.

Nota de citación del dataset (CC BY 4.0 — obligatoria):

Xavier, P., Rodrigues, P., & Silva, C. L. M. (2024). *Hass Avocado Ripening Photographic Dataset* [Data set]. Mendeley Data. <https://doi.org/10.17632/3xd9n945v8.1>

Anexos

Anexo A — Comandos de verificación del sistema en producción

```
# Verificar que la API esté activa y el modelo cargado
curl https://api.aguacatia.warlockcode.com/health

# Clasificar una imagen de aguacate
curl -X POST https://api.aguacatia.warlockcode.com/classify \
  -F "file=@foto_aguacate.jpg" \
  -F "usuario=prueba"

# Consultar historial de clasificaciones
curl "https://api.aguacatia.warlockcode.com/history?limit=10"

# Ver documentación interactiva de la API
# Abrir en navegador: https://api.aguacatia.warlockcode.com/docs
```

Anexo B — Clases del modelo y mapeo interno

Índice (modelo)	Clave interna	Nombre completo	Emo-ji	Color
0	unripe	Unripe — Verde, no apto para consumo		Verde
1	breaking	Breaking — En transición		Amari- llo
2	ripe_first	Ripe First Stage — Casi listo		Naranja
3	ripe_second	Ripe Second Stage — Punto óptimo		Morado
4	overripe	Overripe — Deteriorado		Rojo

Anexo C — Dependencias de producción (API)

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
python-multipart==0.0.12
aiosqlite==0.20.0
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.4.1+cpu
ultralytics==8.4.60
Pillow==10.4.0
boto3==1.35.0
```

Anexo D — Estructura de la base de datos

```
CREATE TABLE IF NOT EXISTS clasificaciones (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  usuario     TEXT NOT NULL,
```

```
imagen_url TEXT NOT NULL,  
clase      TEXT NOT NULL,          -- unripe|breaking|  
         ripe_first|ripe_second|overripe  
confianza  REAL NOT NULL,          -- valor entre 0.0 y 1.0  
fecha_at   TEXT NOT NULL          -- ISO 8601:  
         2026-06-08T14:32:10.123+00:00  
);
```

Fin del documento

**Aguacatia — Universidad de La Salle Maestría en Ciencias de la Computación
María Alejandra Gómez Piedrahita · Juan Manuel Castillo Pinto Bogotá D.C.,
Colombia — Junio 2026**